

#TITLE:

Advanced Dimensionality Reduction & Data Augmentation for Image-Based ML With Fashion MNIST — PCA, t-SNE, UMAP, Albumentations, and Classifier

□ HOW TO USE THIS IN COLAB

Open Google Colab

Start a new notebook

Enable GPU: Runtime → Change runtime type → Hardware Accelerator: GPU

Copy and paste each code block from this .docx file into a new Colab cell

Run the cells and modify as needed!

INSTALL DEPENDENCIES

```
!apt-get update -y
!apt-get install -y build-essential

0% [Working]          Get:1
https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
0% [Waiting for headers] [Waiting for headers] [Connected to
r2u.stat.illinois.
Hit:2 https://cli.github.com/packages stable InRelease
0% [Waiting for headers] [Waiting for headers] [Connected to
r2u.stat.illinois.
Get:3
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/
x86_64 InRelease [1,581 B]
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:5 http://security.ubuntu.com/ubuntu jammy-security InRelease [129
kB]
Get:6 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128
kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127
kB]
Get:9
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/
x86_64 Packages [2,123 kB]
Hit:10 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu
jammy InRelease
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy
```

```

InRelease
Get:12 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy
InRelease [18.1 kB]
Get:13 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages
[2,825 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/main amd64
Packages [3,526 kB]
Get:15 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages
[9,432 kB]
Get:16 http://security.ubuntu.com/ubuntu jammy-security/restricted
amd64 Packages [5,969 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/universe amd64
Packages [1,289 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64
Packages [1,594 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64
Packages [6,168 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64
Packages [3,856 kB]
Get:21 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu
jammy/main amd64 Packages [38.5 kB]
Fetched 407 kB in 6s (71.9 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as
repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does
not seem to provide it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
0 upgraded, 0 newly installed, 0 to remove and 45 not upgraded.

```

```
!pip install -q umap-learn albumentations opentsne torchvision
```

```

_____ 0.0/3.5 MB ? eta -:--:--
_____ 0.9/3.5 MB 26.0 MB/s eta
0:00:01 _____ 3.5/3.5 MB 51.5 MB/s
eta 0:00:01 _____ 3.5/3.5 MB 30.7
MB/s eta 0:00:00

```

```
# ☐ IMPORTS & SETUP
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import torch
import torchvision
from torchvision import transforms
from tensorflow.keras.datasets import fashion_mnist
from sklearn.decomposition import PCA

```

```

from sklearn.manifold import TSNE
import umap.umap_ as umap
from openTSNE import TSNE as OpenTSNE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import warnings
warnings.filterwarnings("ignore")
sns.set(style='whitegrid')

# □ LOAD AND PREPROCESS DATA
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X = X_train.reshape((X_train.shape[0], -1))
X = StandardScaler().fit_transform(X)
y = y_train

label_dict = {
    0: "T-shirt", 1: "Trouser", 2: "Pullover", 3: "Dress", 4: "Coat",
    5: "Sandal", 6: "Shirt", 7: "Sneaker", 8: "Bag", 9: "Ankle boot"
}

Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 _____ 0s 0us/step

# □ PCA: Principal Component Analysis
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

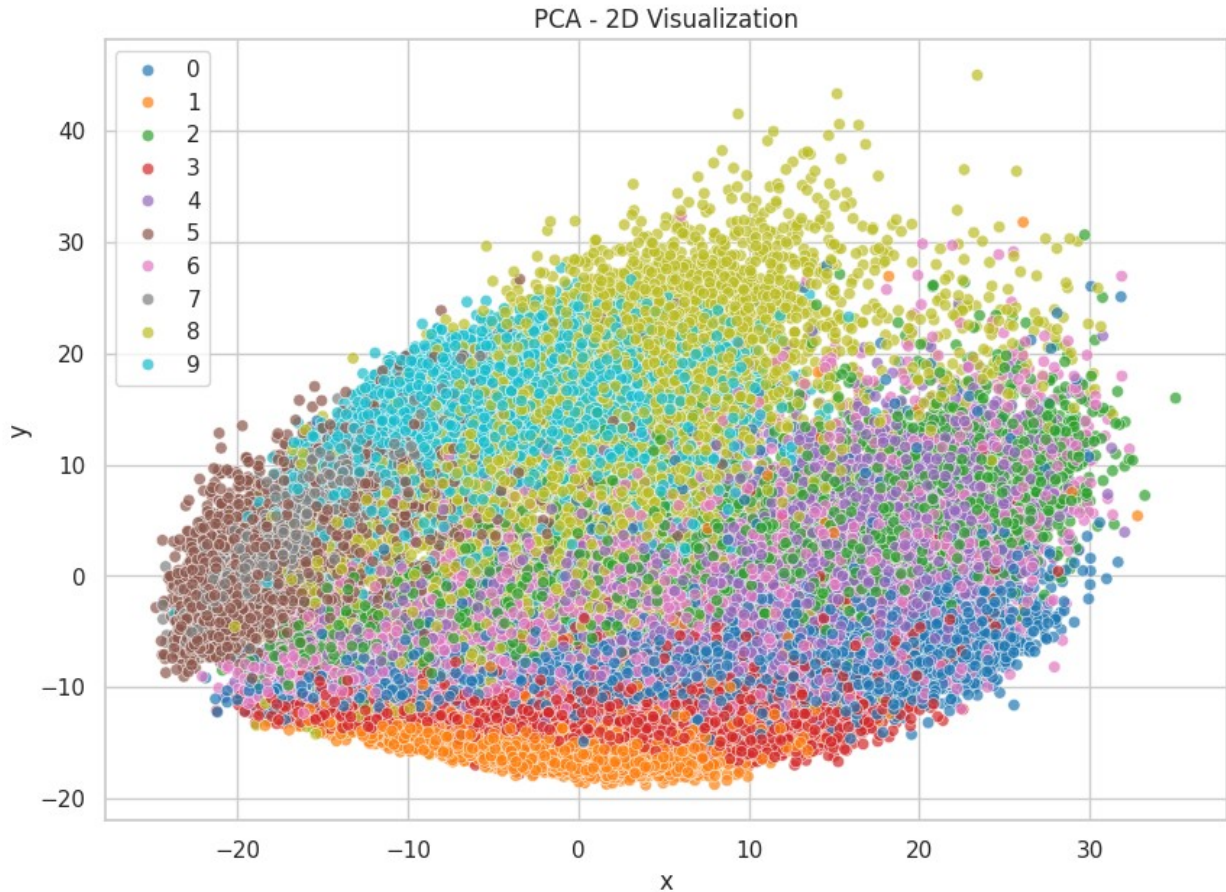
Explained Variance Ratio: [0.22083547 0.14414633]

# □ Visualize
def plot_2D(X_emb, y, title):
    df = pd.DataFrame({'x': X_emb[:, 0], 'y': X_emb[:, 1], 'label':
y})
    plt.figure(figsize=(10, 7))
    sns.scatterplot(data=df, x='x', y='y', hue='label',
palette='tab10', alpha=0.7)
    plt.title(title)

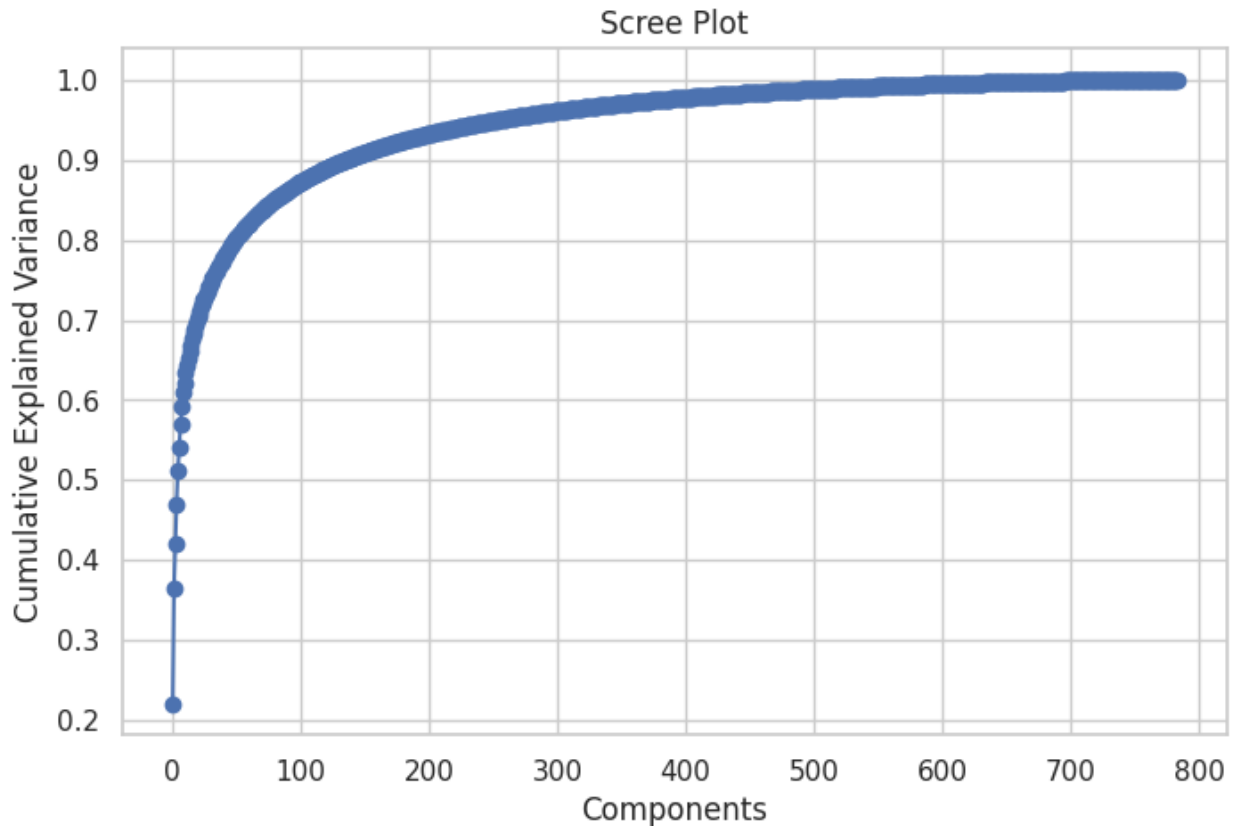
```

```
plt.legend()
plt.show()

plot_2D(X_pca, y, "PCA - 2D Visualization")
```



```
# Scree Plot
pca_full = PCA().fit(X)
plt.figure(figsize=(8, 5))
plt.plot(np.cumsum(pca_full.explained_variance_ratio_), marker='o')
plt.title("Scree Plot")
plt.xlabel("Components")
plt.ylabel("Cumulative Explained Variance")
plt.grid(True)
plt.show()
```



PCA Code Expalanation:

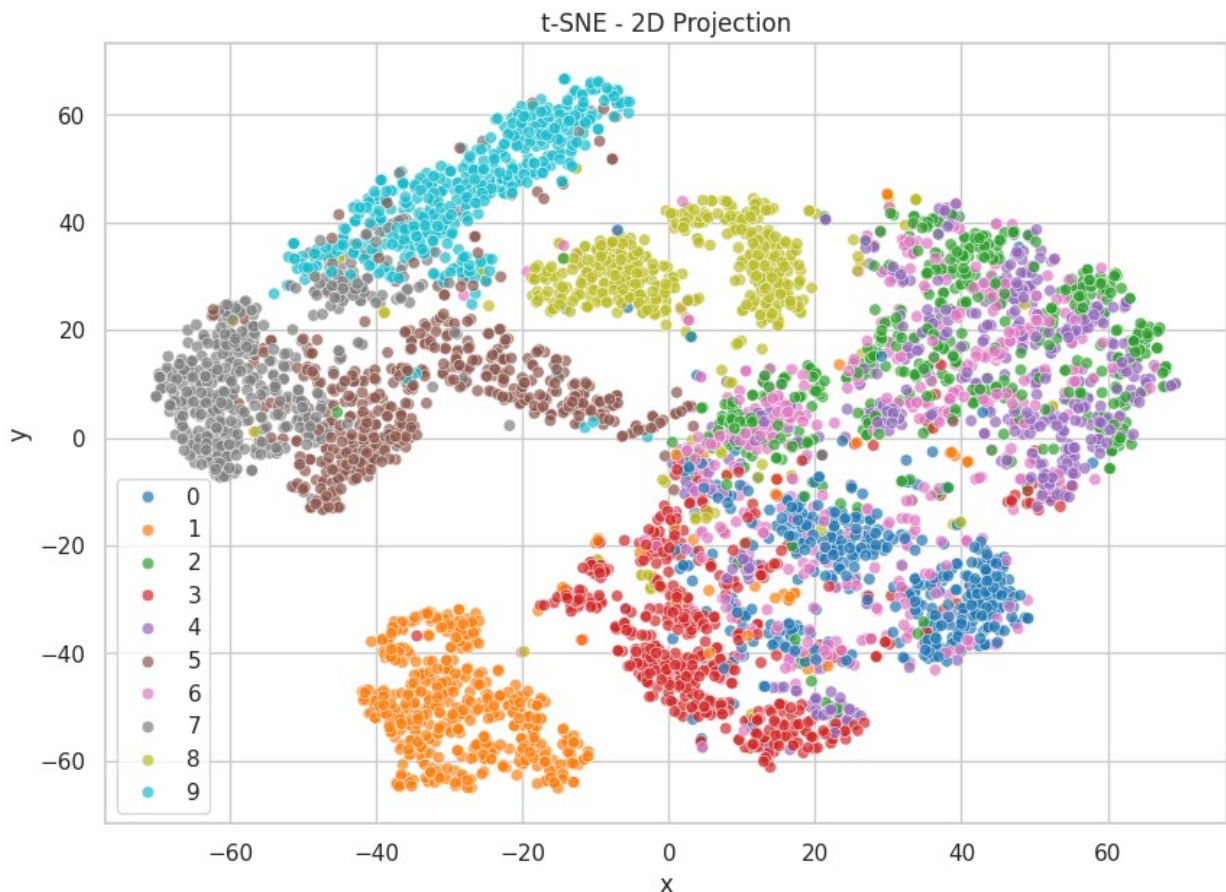
The code starts with the creation of the environment and loading of Fashion-MNIST dataset. The data is composed of 28 grayscale images 28x28 of video images of ten types of clothes. Each picture is then transformed back into a 784 element one dimensional vector to allow numerical operations to be performed on the pictures. The results are then normalized with the help of StandardScaler to make each pixel feature mean zero and variance one that is significant since PCA is sensitive to differences in scales. Only in the case of result visualization, a label dictionary is defined to be readable. PCA will use two components and this implies that the original high dimensional data will be projected onto two dimensional space although it attempts to capture the maximum possible variance. The ratio of explained variance printed following PCA shows the percentage of the important information within the data retained in the two components.

The visualization of the output of PCA is done by a custom function plot2D. It draws a scatter plot with every point being an image and coloured in its own true label and you can see how various types of clothes pack together in lower dimensional space. The resultant Scree Plot below is the product of pca that is not limited with regard to components. It demonstrates the growth of cumulative explained variance of the addition of additional principal components. When the curve is increasing and then it becomes horizontal, it implies that a comparatively few key elements are used to capture the greatest portion of the beneficial design in the data. This is used to determine the number of components to retain in the process of dimensionality reduction to be used in other downstream tasks like classification or visualization.

```
# □ t-SNE with openTSNE (GPU-Friendly)
X_sample = X[:5000]
y_sample = y[:5000]

tsne = OpenTSNE(n_components=2, perplexity=30, n_jobs=4)
X_tsne = tsne.fit(X_sample)

plot_2D(X_tsne, y_sample, "t-SNE - 2D Projection")
```

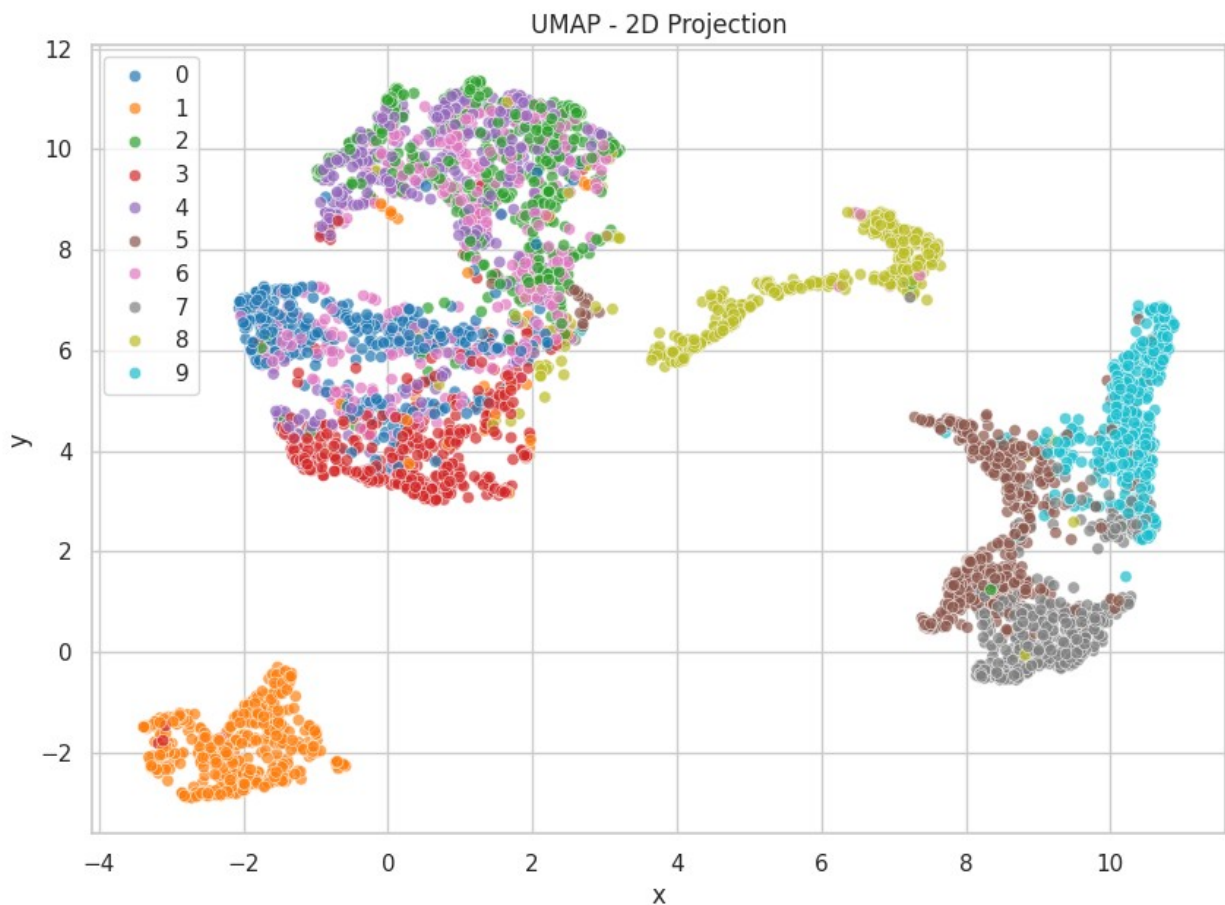


T-SNE Code Expalnation:

The t-SNE is used in this code to project a dataset onto 2 dimensions to visualize it. As t-SNE is computationally expensive, only 5,000 images, in the first place, are employed. The implementation of the OpenTSNE makes the process faster and the parameter perplexity=30 can be used to balance the way local neighborhoods are created. Once 2D embedding (Xtsne) has been computed, plot2D function is used to create a scatter plot in which the color of every point is a class name, the visual effects of which we can see and visualize how various Fashion-MNIST categories are grouped together.

```
# □ UMAP
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1)
X_umap = reducer.fit_transform(X_sample)
```

```
plot_2D(X_umap, y_sample, "UMAP - 2D Projection")
```



Code Expalantion:

This code uses UMAP in order to compress the information and visualize it in 2 dimensions. UMAP constructs a graph of closest similarities in the high dimension space and subsequently projects it to a lower dimension layout maintaining the structure. The parameter `nneighbors=15` is used to control the amount of local versus global structure to retain, and `mindist=0.1` used to control the tightness of the point clustering. The resultant embedding `Xumap` is plotted using `plot2D` and the similar images cluster together in the 2D space

```
# DATA AUGMENTATION
from PIL import Image
import albumentations as A
import cv2

# Sample images for augmentation
sample_images = X_train[:5]
pil_images = [Image.fromarray(img) for img in sample_images]

# torchvision transforms
```

```

transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=1.0),
    transforms.RandomRotation(30),
    transforms.RandomResizedCrop(28, scale=(0.8, 1.0)),
    transforms.ColorJitter(brightness=0.5, contrast=0.5)
])

def show_augmented(images, transform_fn, title):
    fig, axs = plt.subplots(1, len(images), figsize=(15, 3))
    for i, img in enumerate(images):
        axs[i].imshow(transform_fn(img), cmap='gray')
        axs[i].axis('off')
    plt.suptitle(title)
    plt.show()

show_augmented(pil_images, transform, "Torchvision Basic
Augmentations")

```

Torchvision Basic Augmentations



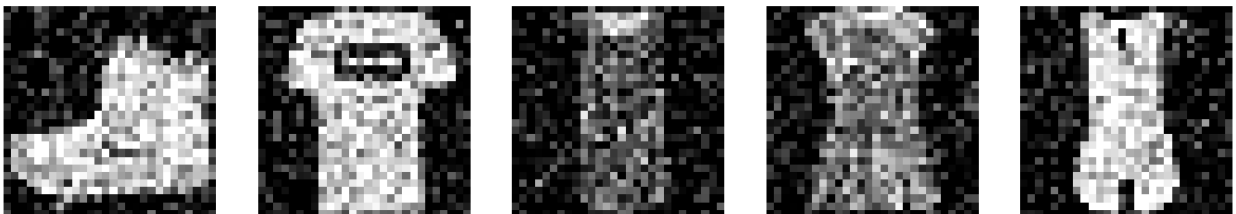
```

# □ Gaussian Noise
def add_gaussian_noise(img, std=0.2):
    noise = np.random.normal(0, std, (28, 28))
    noisy = np.clip(np.array(img)/255.0 + noise, 0, 1)
    return Image.fromarray((noisy * 255).astype(np.uint8))

show_augmented(pil_images, add_gaussian_noise, "Gaussian Noise
Augmentation")

```

Gaussian Noise Augmentation



```

# □ Albumentations Pipeline
alb_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=40, p=0.8),

```

```

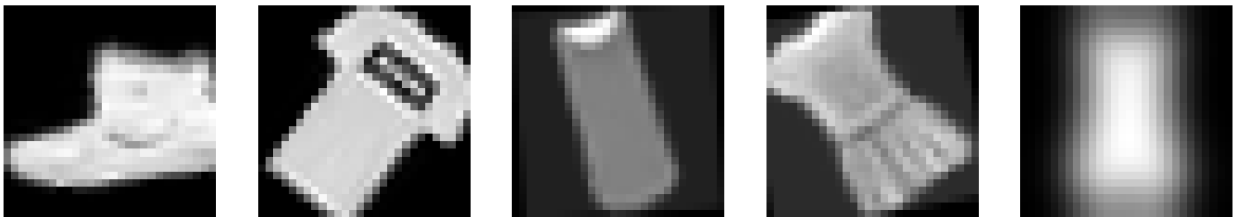
    A.RandomBrightnessContrast(p=0.5),
    A.ShiftScaleRotate(shift_limit=0.0625, scale_limit=0.2,
rotate_limit=15, p=0.8),
    A.GaussianBlur(p=0.2)
])

def apply_albumentations(images, transform):
    fig, axs = plt.subplots(1, len(images), figsize=(15, 3))
    for i, img in enumerate(images):
        aug = transform(image=np.array(img))["image"]
        axs[i].imshow(aug, cmap='gray')
        axs[i].axis('off')
    plt.suptitle("Albumentations – Advanced Augmentations")
    plt.show()

apply_albumentations(pil_images, alb_transform)

```

Albumentations – Advanced Augmentations



```

# □ CLASSIFIER ON REDUCED FEATURES
X_reduced = PCA(n_components=50).fit_transform(X[:10000])
y_reduced = y[:10000]

clf = LogisticRegression(max_iter=2000)
score = cross_val_score(clf, X_reduced, y_reduced, cv=5)
print("Accuracy (PCA-50):", score.mean())

Accuracy (PCA-50): 0.8355

```

Code Expalnation:

In this part of the code, the various data augmentation methods are illustrated in order to enhance the visual diversity of the training data. One, a small collection of pictures is chosen and transformed into PIL format to be able to easily apply augmentations. The pipeline of TorchVision augmentation is assigned which consists of horizontal flipping, random rotation, random cropping and resizing, and brightness/contrast modifications. These processes will be operated on every image of the sample and the output will be presented using showaugmented. The reason is to represent the natural variations and the model is more resistant to the orientation, lighting and shape changes. Then, a tailor-made Gaussian noise filter is used, and random noise is introduced to the picture. This assists this model to be able to train on grainy or low quality inputs. The code then adds an Albumentations pipeline that makes more sophisticated and adaptive transformations to further variation of data. The

applyalbumentations visualizes the roles concerning how these transformations alter the sample images.

Lastly, a code does classification of PCA-reduced data. PCA is used to reduce the 50 major components of the dataset by compressing the data without losing most of the significant variance in the data. The reduced dataset is trained on the Logistic Regression using 5-fold cross-validation. The score on accuracy indicates the performance of a simple classifier when using the compressed features representations. This illustrates the fact that dimensionality reduction is a technique that helps in simplifying the computations whilst still maintaining good classification.

#QUESTIONS

□ PCA

Coding:

Train a classifier using PCA(100) components.

Compare performance with PCA(10).

Visualize PCA-transformed images in 3D.

Theory:

What does PCA optimize?

When can PCA hurt model performance?

```
#train a classifier using PCA(100) components.
# Use a manageable subset for speed
N = 10000
X_sub = X[:N]
y_sub = y[:N]

# Fit PCA with 100 components on the subset
pca100 = PCA(n_components=100, random_state=42)
X_pca100 = pca100.fit_transform(X_sub)

# Train & evaluate a Logistic Regression classifier on PCA(100)
features
clf100 = LogisticRegression(max_iter=2000, n_jobs=-1)
scores100 = cross_val_score(clf100, X_pca100, y_sub, cv=5)

print(f"PCA(100) → Logistic Regression accuracy (cv=5) on first {N}
samples:")
print(f"  Mean: {scores100.mean():.4f} | Std: {scores100.std():.4f}")

# (Optional) Fit on full subset to keep a trained model for later use
clf100.fit(X_pca100, y_sub)
```

```
PCA(100) → Logistic Regression accuracy (cv=5) on first 10000 samples:  
Mean: 0.8370 | Std: 0.0070
```

```
LogisticRegression(max_iter=2000, n_jobs=-1)
```

Expalantion:

This code is used to train a Regression Logistic classification on features that have been reduced with the help of PCA to 100 components. As using the entire dataset may be time-consuming, the initial 10,000 samples are used to train the model (Xsub and ysub). This is followed by the application of PCA with ncomponents=100 implying that the original 784-quantity of pixels of the image are reduced to 100 new features that represent a bulk of the important structure of the dataset. This eliminates noise, dimensionality reduction, and the training time is reduced but useful patterns to classify data are still retained. The modified data is saved in X_pca100 that serves as input of the classifier.

A Logistic regression model is then trained and tested on these PCA transformed features by 5 fold cross-validation. This is a way of testing the effectiveness of the model in generalization; it is done by training and testing the model on several splits of the dataset. The standard deviation indicates consistency in the folds and the printed mean accuracy is the overall performance estimate. Lastly, the model is again fitted on the full reduced dataset to retain an end trained model. On the whole, this code demonstrates that dimensionality reduction in PCA may enable a classifier to work fine but improves its efficiency and minimizes its cost of computation.

```
# PCA(10) → Logistic Regression (5-fold CV)  
  
# Use the same subset (X_sub, y_sub) already defined above  
  
pca10 = PCA(n_components=10, random_state=42)  
X_pca10 = pca10.fit_transform(X_sub)  
  
clf10 = LogisticRegression(max_iter=2000, n_jobs=-1)  
scores10 = cross_val_score(clf10, X_pca10, y_sub, cv=5)  
  
print(f"\nPCA(10) → Logistic Regression accuracy (cv=5) on first  
{len(y_sub)} samples:")  
print(f" Mean: {scores10.mean():.4f} | Std: {scores10.std():.4f}")
```

```
# Fit full subset model for consistency  
clf10.fit(X_pca10, y_sub)
```

```
PCA(10) → Logistic Regression accuracy (cv=5) on first 10000 samples:  
Mean: 0.7460 | Std: 0.0062
```

```
LogisticRegression(max_iter=2000, n_jobs=-1)
```

```
print("\n Performance comparison of the PCA 100 and PCA 10")  
print("-----")  
print(f"PCA(100) Accuracy (mean ± std): {scores100.mean():.4f} ±  
{scores100.std():.4f}")
```

```
print(f"PCA(10) Accuracy (mean ± std): {scores10.mean():.4f} ± {scores10.std():.4f}")
```

Performance comparison of the PCA 100 and PCA 10

```
-----
PCA(100) Accuracy (mean ± std): 0.8370 ± 0.0070
PCA(10) Accuracy (mean ± std): 0.7460 ± 0.0062
```

Code Expalnation:

By using this code, the dataset is reduced to 10 PCA components and a Logistic Regression model is trained with the compressed features. Since PCA(10) retains significantly less information than PCA(100), the classifier will have fewer patterns of meaning to study. The results of the model are tested with the 5-fold cross-validation and the average accuracy and variability are printed. Lastly, the findings are contrasted with the previous model PCA(100) to demonstrate that dimensional reduction may adversely affect performance in case it is performed too heavily.

```
# 3D Visualization of PCA-transformed images – continuation

# Use the same subset to stay consistent (and a smaller slice for
plotting speed)
X_vis = X_sub
y_vis = y_sub
vis_n = 3000 # reduce points for a clearer, faster 3D plot
X_vis_small = X_vis[:vis_n]
y_vis_small = y_vis[:vis_n]

# Fit 3-component PCA and transform
pca3 = PCA(n_components=3, random_state=42)
X_pca3 = pca3.fit_transform(X_vis_small)

# 3D scatter plot colored by label
from mpl_toolkits.mplot3d import Axes3D # noqa: F401 (needed for 3D
projection)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

# Map labels to colors using tab10
cmap = plt.get_cmap("tab10")
colors = [cmap(int(lbl) % 10) for lbl in y_vis_small]

ax.scatter(X_pca3[:, 0], X_pca3[:, 1], X_pca3[:, 2], c=colors, s=10,
alpha=0.7)

ax.set_title("PCA (3D) – Fashion-MNIST (first {}
samples)".format(vis_n))
ax.set_xlabel("PC1 {:.2f}%")
```

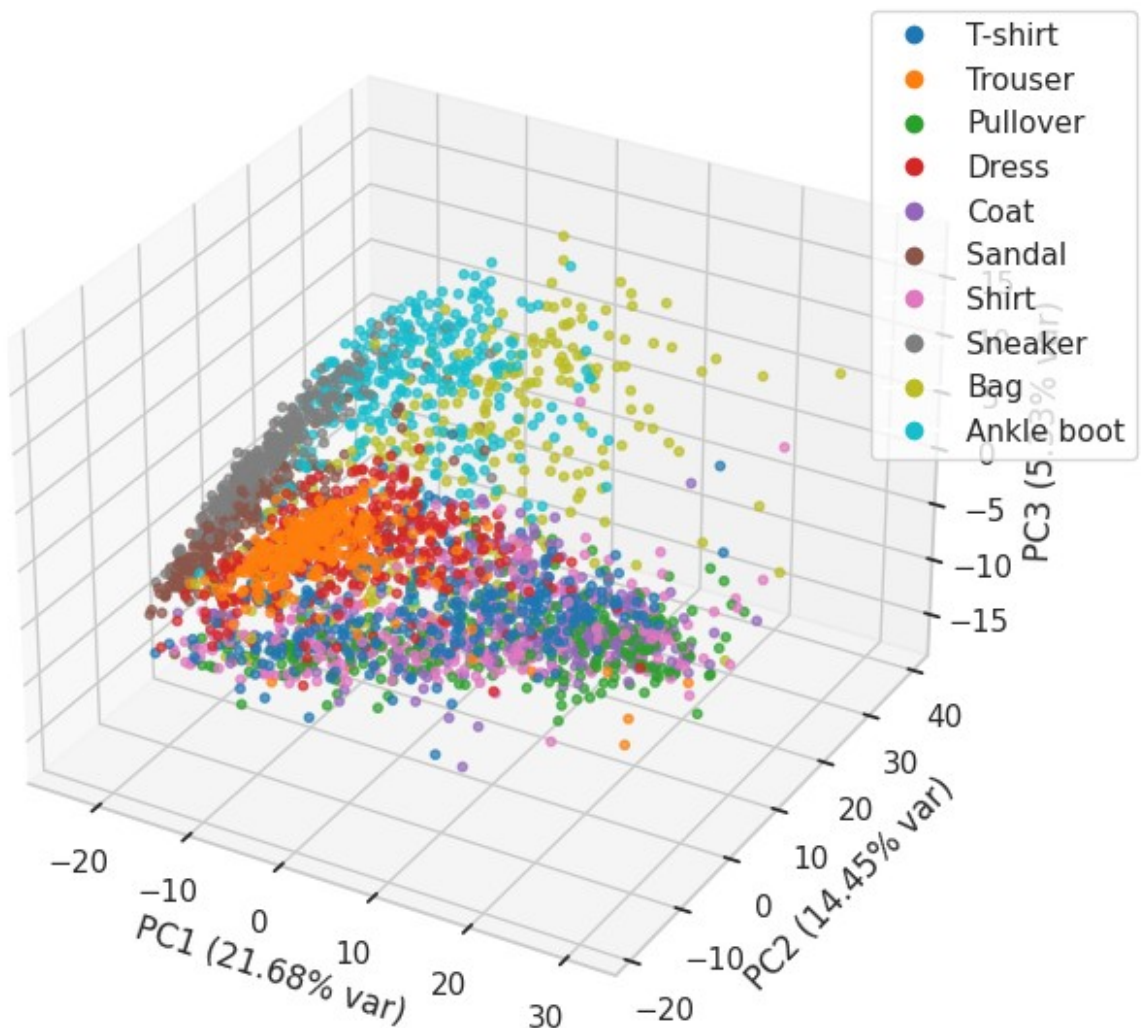
```

var)".format(pca3.explained_variance_ratio_[0]*100))
ax.set_ylabel("PC2 ({:.2f}%
var)".format(pca3.explained_variance_ratio_[1]*100))
ax.set_zlabel("PC3 ({:.2f}%
var)".format(pca3.explained_variance_ratio_[2]*100))

# Create a compact legend using label_dict
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=label_dict[k],
           markerfacecolor=cmap(k), markersize=8)
    for k in sorted(label_dict.keys())
]
ax.legend(handles=legend_elements, loc='upper right',
bbox_to_anchor=(1.15, 1.0))
plt.show()

```

PCA (3D) — Fashion-MNIST (first 3000 samples)



Expalnation:

This code is used to visually plot 3D PCA -transformed features. It initially narrows down to 3,000 samples to plot it faster and easier to understand. Then it uses 3-component PCA which, instead of using 784 features in each image, uses only three numbers which reflect the most significant patterns in the data. The x, y and z coordinates are applied as these three major components of a 3D scatter plot. One image is paired with one point in the plot, and the points are colored as per their label in the classes to be able to see visually how various categories of clothing are lowered in less space. The axis labels also indicate the amount of variance that each principal component accounts which gives an idea of the extent to which significant information is captured along the similar direction. There is a legend attached to conform each color to its type of clothing. With the aid of this visualization, we can realize that various classes can be separated in case they are pushed into three dimensions.

Q. What is PCA?

Principal Component Analysis (PCA) is aimed at converting high-dimensional data to a smaller set of features, as well as preserving as much of the original information as possible. The main concept of PCA is that not every dimension of the data will add the same value to the knowledge of the structure or patterns the data has. Certain directions in the data are characterized by large variation whereas others are a result of noise or small fluctuations. PCA determines the direction of the highest variance and form the new axes, referred to as the principal components, that represent the most significant patterns, in decreasing order of significance. The first component captures the highest share of the variance, the second captures the second largest share and is orthogonal to the first one, and this goes on to all the components. PA finds the most informative structure by only choosing the first few components, which diminish the dimensionality of the data. The optimization is basically making an attempt to present the data in a limited region with minimal loss of information which is measured by the amount of variance retained. This renders PCA applicable in simplifying data sets, reducing storage and computation, visualizing complex data and as a preprocessing stage to machine learning models which are weak with high-dimensional input.

Q.When does PCA hurts the model?

Even though PCA is not necessarily a bad practice, blindly implementing it may lead to a decline in performance in ways that are not always immediately evident. PCA makes an assumption that the most significant patterns of the data are explained by the directions with the highest variance, which is not always the case with the classification or prediction problems. In some cases, distinguishing features of classes are of low variance, and they are not so evident in the dataset. When PCA is applied to dimensionality reduction, these small, yet significant differences can be eliminated and the model is deprived of potentially useful discriminative information. PCA is also unstable in that it combines the characteristics into new combinations that could decrease interpretability in the event that the original characteristics mattered. In case of too few components one could likely end up with an excessively simplistic compressed representation that cannot be used to perform proper classification. PCA is not as useful when the dataset is already small as well as when feature-feature relationships are non-linear due to the linear nature of PCA. Such strategies as t-SNE or UMAP can help conserve the neighborhood and class associations in such instances. Thus, PCA is to be applied consciously of the task to be done and experimented with varying number of components to make sure that it facilitates but does not inhibit performance.

□ t-SNE

Coding:

Try different perplexities (5, 30, 50).

Apply t-SNE on PCA-reduced data.

Use t-SNE for only two classes.

Theory:

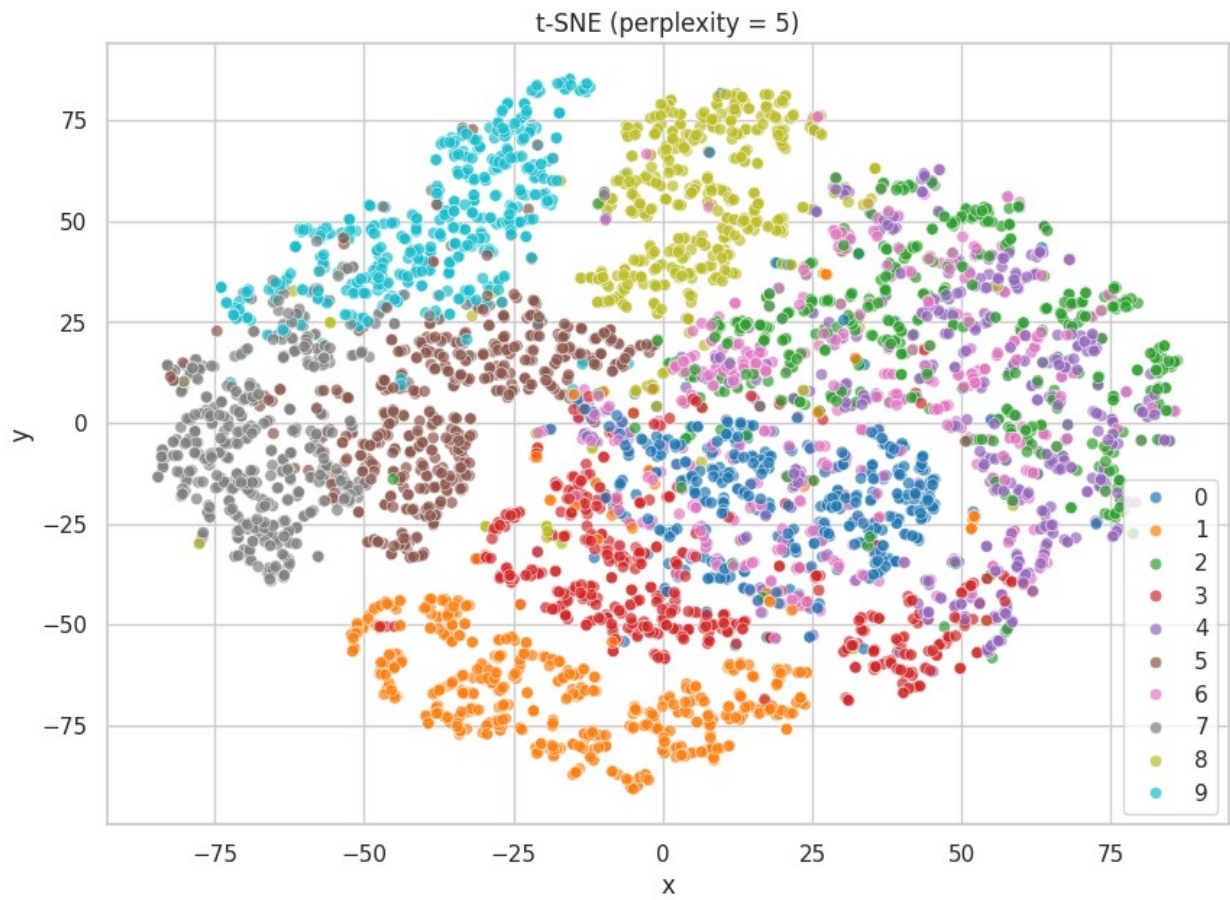
Why is t-SNE bad for classification?

What does perplexity control?

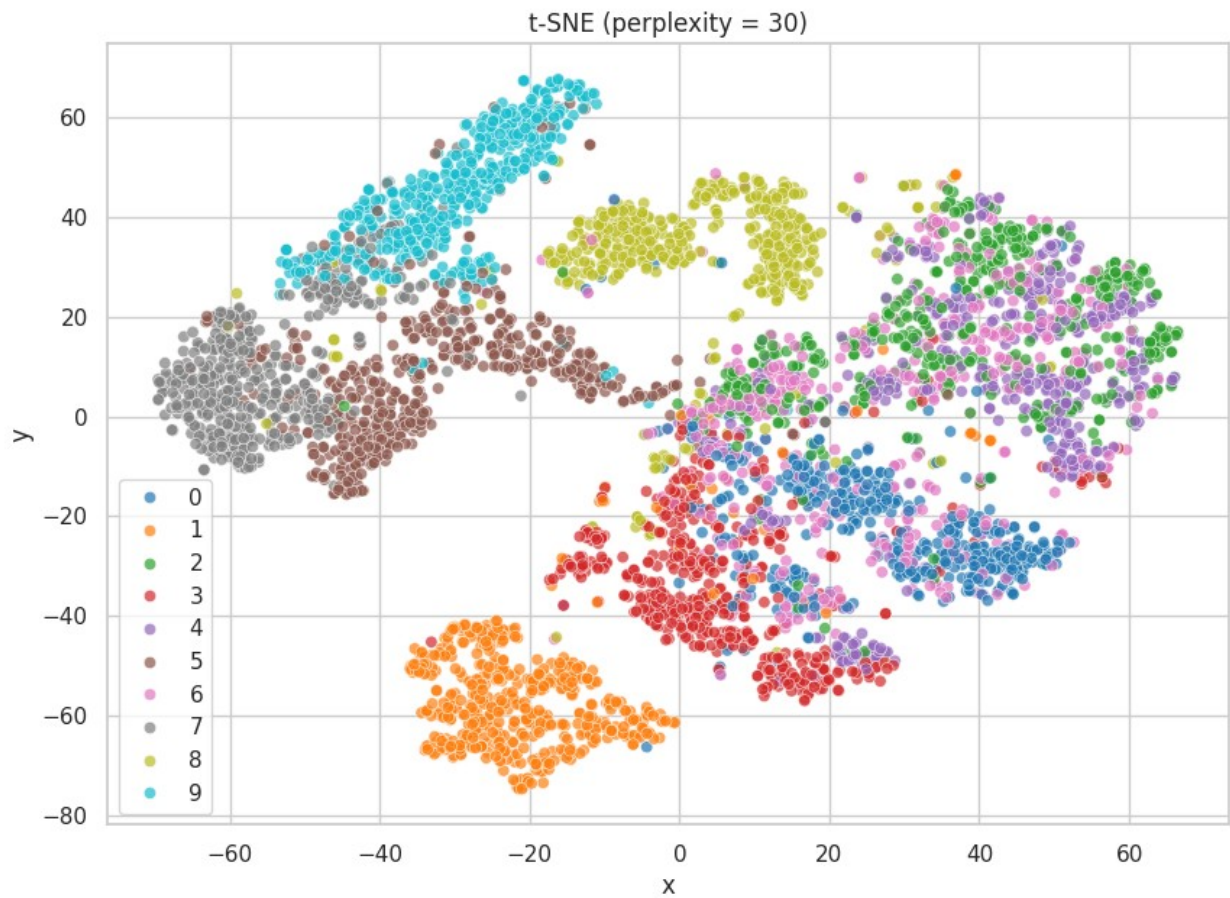
```
#Trying different perplexities
perplexities = [5, 30, 50]
tsne_results = {}

for p in perplexities:
    print(f"\nRunning t-SNE with perplexity = {p} ...")
    tsne_model = OpenTSNE(n_components=2, perplexity=p, n_jobs=4,
random_state=42)
    X_tsne_p = tsne_model.fit(X_sample)
    tsne_results[p] = X_tsne_p
    plot_2D(X_tsne_p, y_sample, f"t-SNE (perplexity = {p})")
```

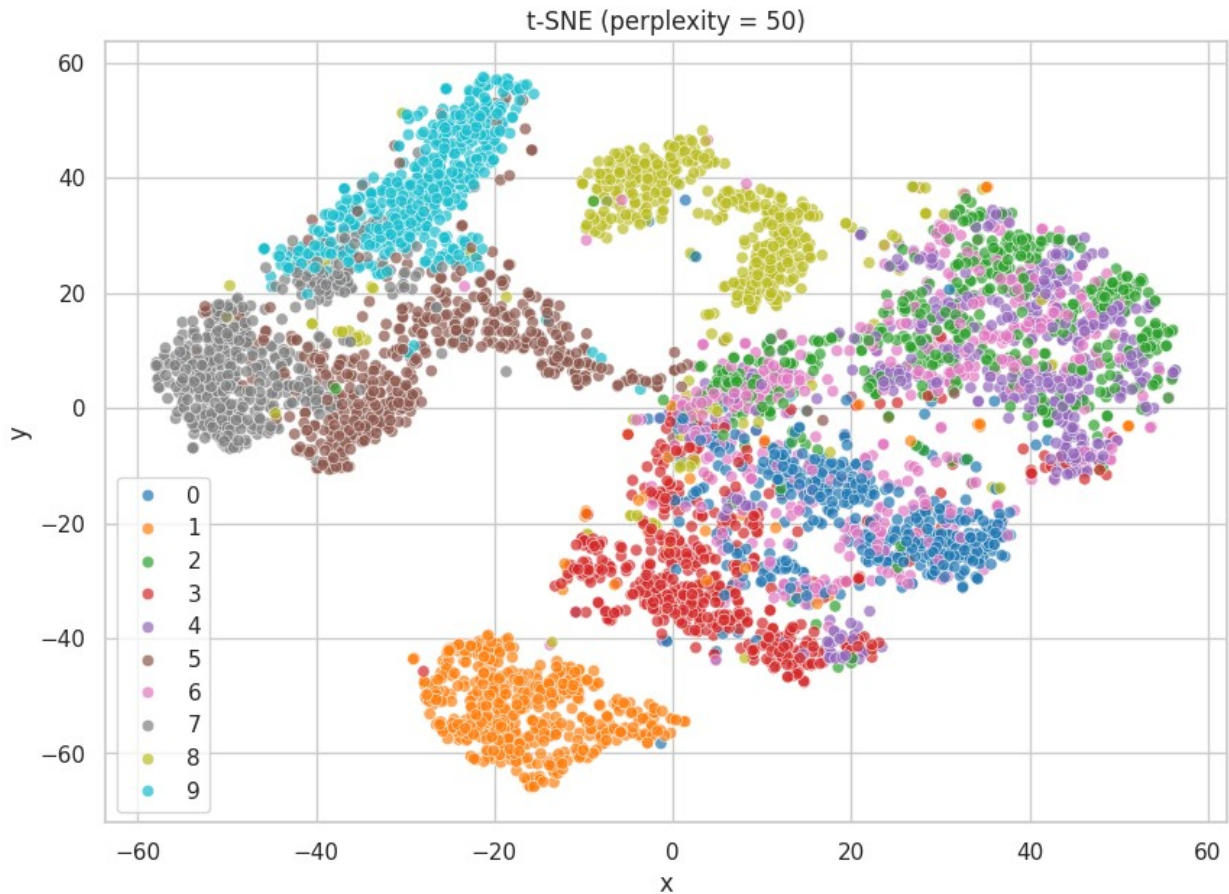
Running t-SNE with perplexity = 5 ...



Running t-SNE with perplexity = 30 ...



Running t-SNE with perplexity = 50 ...



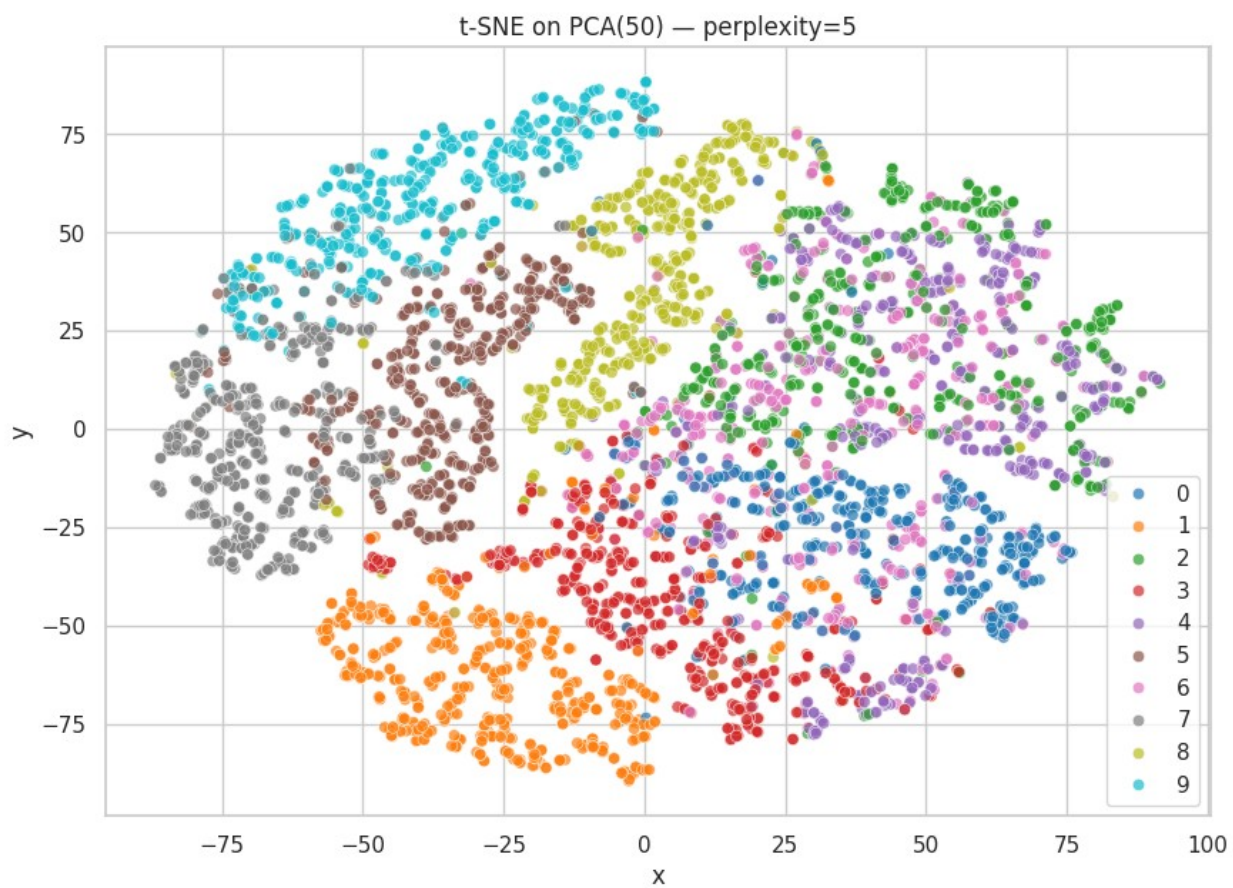
```
# t-SNE on PCA-reduced data
```

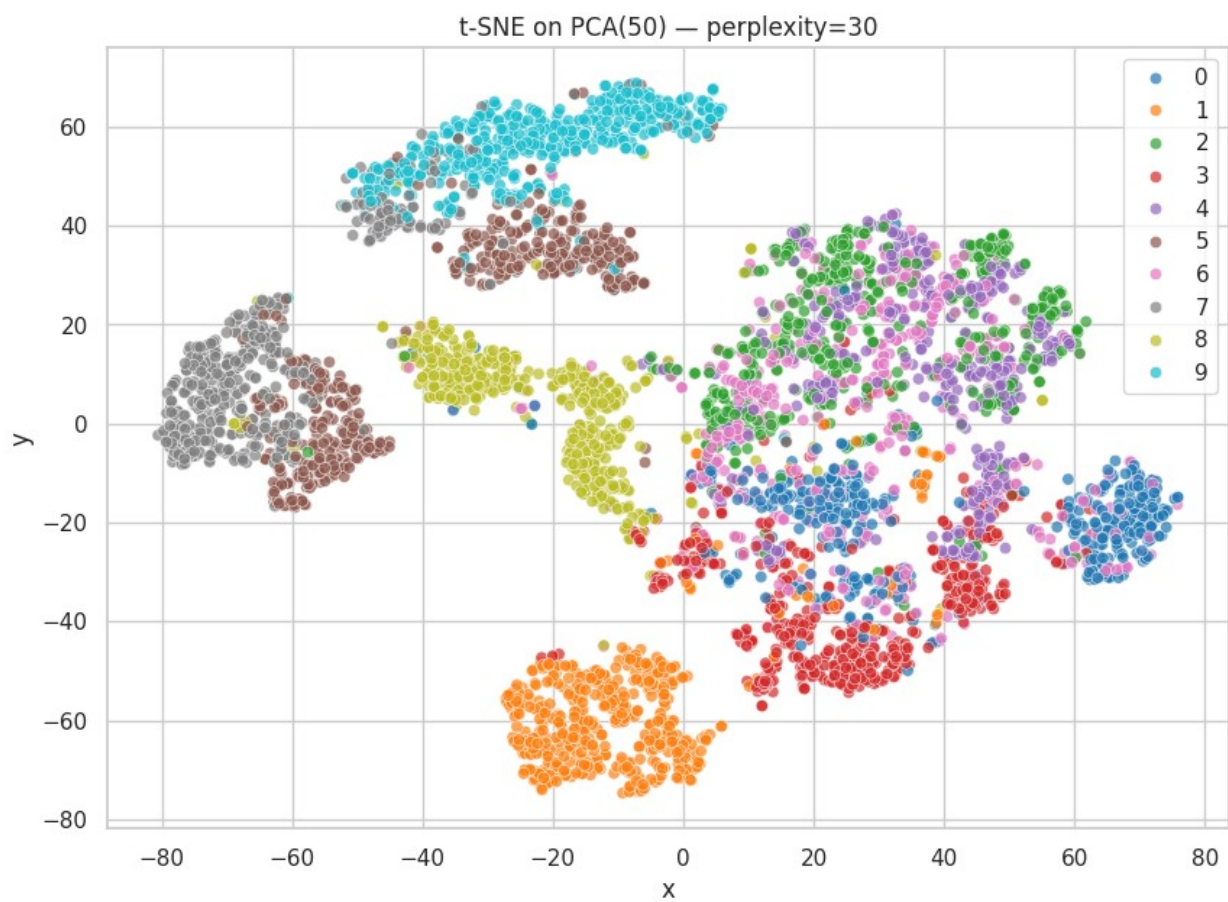
```
# 1) Reduce dimensionality with PCA first (helps speed & denoise for t-SNE)
```

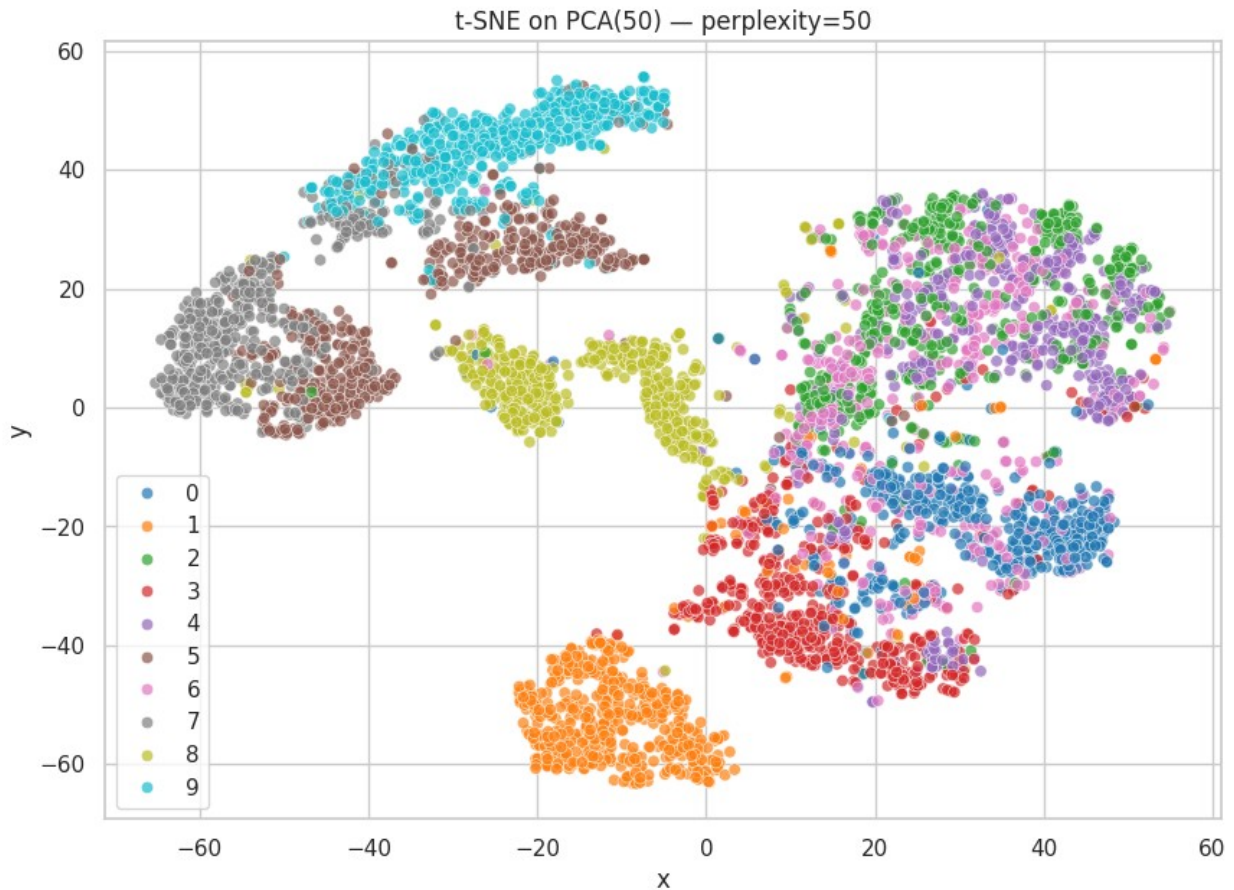
```
pca_for_tsne = PCA(n_components=50, random_state=42)
X_pca50_sample = pca_for_tsne.fit_transform(X_sample)
```

```
# 2) Run t-SNE on the PCA(50) features with a few perplexities
```

```
for p in [5, 30, 50]:
    tsne_p = OpenTSNE(
        n_components=2,
        perplexity=p,
        n_jobs=4,
        random_state=42,
        initialization="pca"
    )
    X_tsne_p = tsne_p.fit(X_pca50_sample)
    plot_2D(X_tsne_p, y_sample, f"t-SNE on PCA(50) – perplexity={p}")
```







```
# t-SNE on only two selected classes

# Choose two classes (you may change 3 and 7 to any other pair: 0-9)
class_a = 3 # Dress
class_b = 7 # Sneaker

# Filter dataset to only these two classes (use same sample size
# logic)
mask = np.isin(y, [class_a, class_b])
X_two = X[mask][:3000] # limit for speed
y_two = y[mask][:3000]

print(f"Selected classes: {label_dict[class_a]} and
{label_dict[class_b]}")
print("Dataset shape:", X_two.shape)

# Optional PCA reduction before t-SNE (recommended)
pca_50_two = PCA(n_components=50, random_state=42)
X_two_pca50 = pca_50_two.fit_transform(X_two)

# Run t-SNE on the filtered data
tsne_two = OpenTSNE(
```

```

n_components=2,
perplexity=30,
n_jobs=4,
random_state=42,
initialization="pca"
)

X_two_tsne = tsne_two.fit(X_two_pca50)

# Plot using the same existing function
plot_2D(X_two_tsne, y_two, f"t-SNE (Only Two Classes:
{label_dict[class_a]} vs {label_dict[class_b]}")

Selected classes: Dress and Sneaker
Dataset shape: (3000, 784)

```



Expalnation:

This code illustrates the change of t-SNE visualizations in various settings and subsets of data. To demonstrate the influence of the different perplexity values on the clustering behavior and preservation of neighborhoods in the 2D projection, t-SNE is initially run with different values of perplexity (5, 30, 50). Subsequently, to increase the efficiency and decrease noise, PCA is used to select the data to 50 components and repeat t-SNE, which tends to be computationally faster

and better cluster separation. Lastly, a dataset is narrowed down to two specific classes (Dress and Sneaker), and t-SNE is executed once more after a reduction using PCA in order to demonstrate how well the algorithm is capable of differentiating between two different classes (specifically, two). The combination of these plots depicts the effect of perplexity on local and global structure, the enhancement of t-SNE stability in cases of preprocessing with the PCA, and the separation of selected classes that is shown by t-SNE.

Q. Why t-SNE is Bad for Classification

t-SNE is an impressive visualization tool, however, it is not applicable to creating or training classifiers. This is due to the fact that t-SNE is maximised on maintaining local similarities i.e. it is concentrated on keeping the points near each other when they are similar without necessarily ensuring that the world structure is significant and consistent. There are two t-SNE plots of the identical data which can be quite different based on the parameters like perplexity, learning rate, initiation, and random seeds. This instability renders the findings to be non-reproducible and unreliable to models which require constant input structure. Also, t-SNE fails to generate an operating transformation that is readily applied to emerging data points. Every new sample would need to be re-embedded with a t-SNE and this is computationally expensive and would be used in large scale classification systems that require real time performance. When doing classification tasks we would prefer features that are stable and predictive but t-SNE only considers visualization aesthetics. It can induce separation in the plotted clusters where the original data may not be separated resulting in misleading interpretation. Thus, t-SNE is not a bad feature representation to train classifiers despite its great capability to explore patterns and gain intuition.

Q. What Perplexity Controls

The most important parameter of t-SNE is perplexity, which regulates the local and global organization in the formation of the visualization 2D or 3D. It has an effect on the definition of the neighborhood size of each data point by the algorithm. Low perplexity value causes the algorithm to emphasize on very close neighbors, which tend to form numerous small tight clusters. This may turn visualization into a fragment and make it too noise-sensitive. The larger the perplexity value, the more distant the points become related and, consequently, the finer distinctions between points may be masked, and the clusters may be mixed. Simply, the perplexity acts as a smoothness adjustment of the visualization. A practical range of between 5 and 50 is usually effective, yet the optimum value lies on data size and density. Less data usually needs less perplexity whereas bigger data usually needs more perplexity negating false cluster shapes. The fact that perplexity remains unchanged in classification accuracy is due to the fact that t-SNE is not a classifier it simply changes the way the data looks in lower dimensions. Perplexity knowledge can be used to make sure that the visualization displays actual patterns rather than visual artifacts.

□ UMAP

Coding:

Change `n_neighbors` and observe changes.

Use UMAP output for KMeans clustering.

Compare UMAP with PCA visually.

Theory:

How does UMAP preserve global structure?

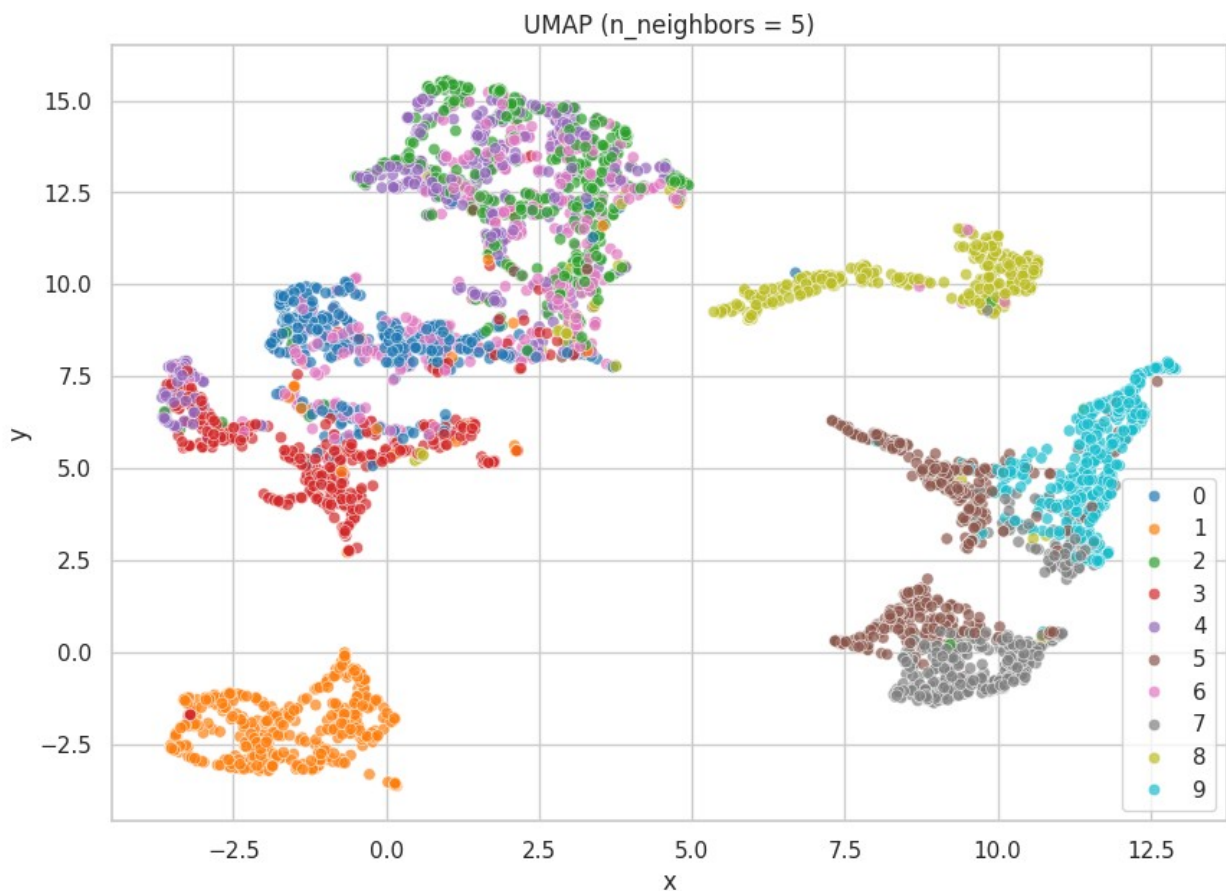
Why is UMAP faster than t-SNE?

```
# ► UMAP with different n_neighbors – continuation
```

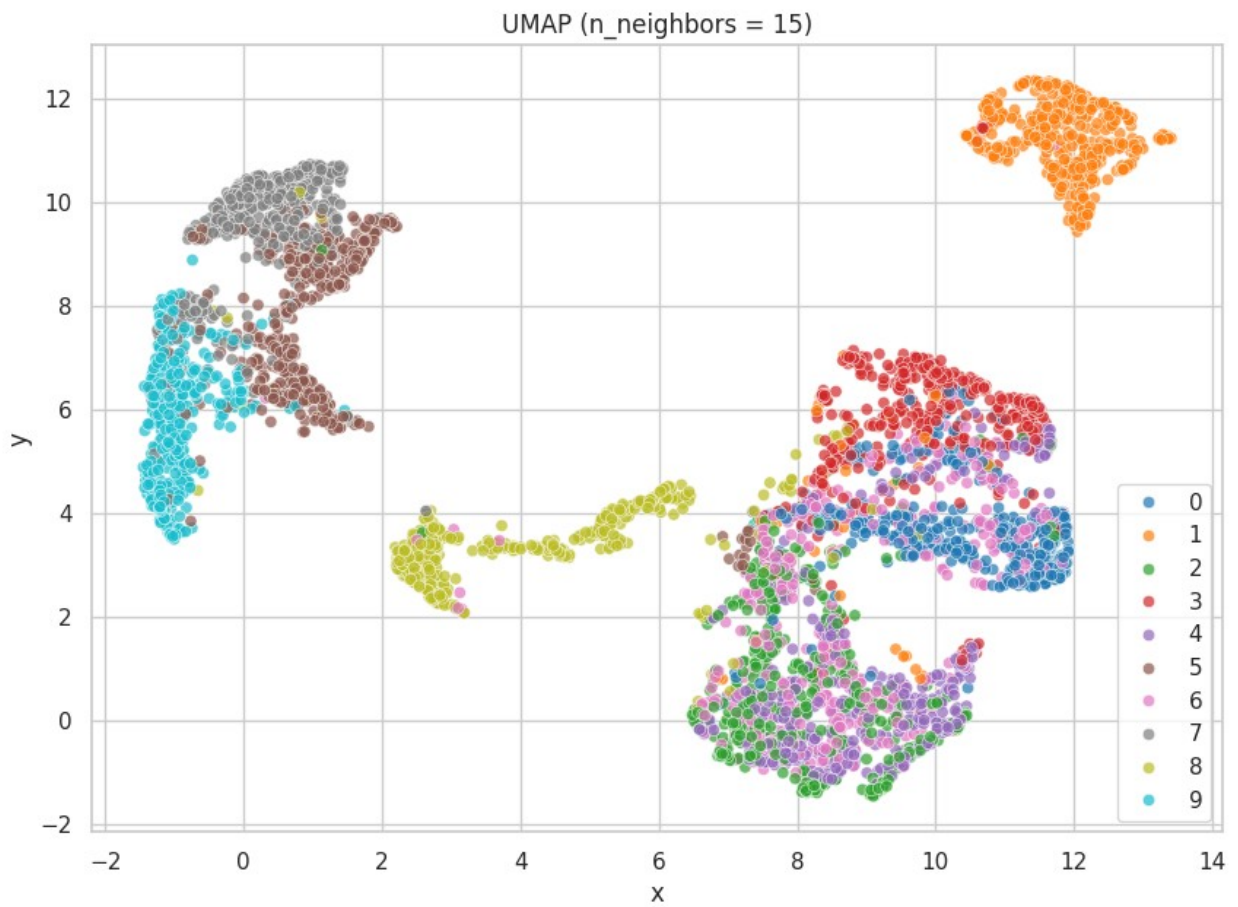
```
neighbor_values = [5, 15, 50]
```

```
for n in neighbor_values:
    print(f"\nRunning UMAP with n_neighbors = {n} ...")
    reducer_n = umap.UMAP(n_neighbors=n, min_dist=0.1,
random_state=42)
    X_umap_n = reducer_n.fit_transform(X_sample)
    plot_2D(X_umap_n, y_sample, f"UMAP (n_neighbors = {n})")
```

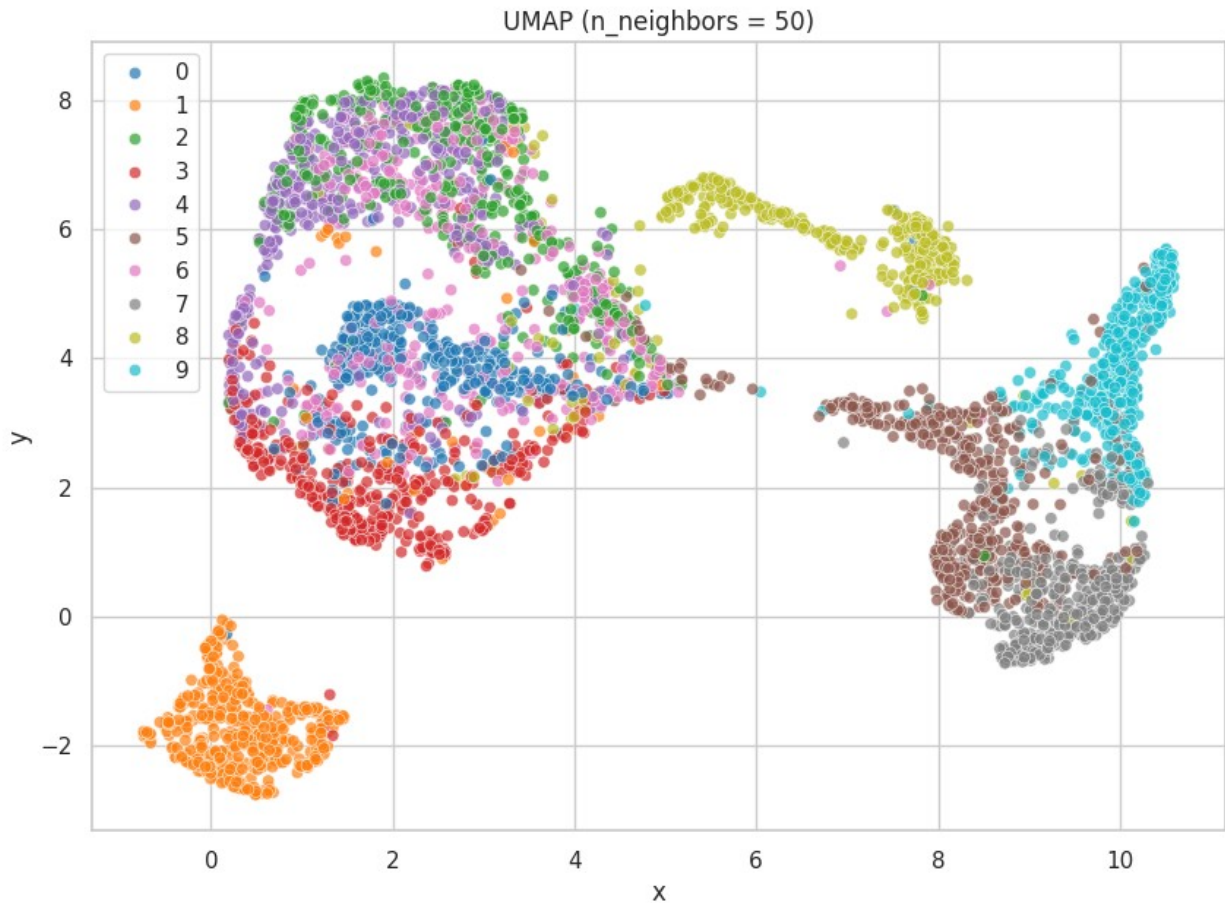
Running UMAP with n_neighbors = 5 ...



Running UMAP with `n_neighbors = 15` ...



Running UMAP with `n_neighbors = 50` ...



► UMAP embedding → KMeans clustering

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score, silhouette_score
import numpy as np

# Use the existing UMAP embedding if available; otherwise compute it
on X_sample
try:
    X_umap
except NameError:
    # Fallback: compute UMAP on the same sample used earlier
    reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X_sample)

# --- KMeans on UMAP space ---
k = 10 # Fashion-MNIST has 10 classes
kmeans = KMeans(n_clusters=k, n_init=20, random_state=42)
cluster_labels = kmeans.fit_predict(X_umap)

# --- Clustering quality metrics ---
```

```

ari = adjusted_rand_score(y_sample, cluster_labels)
nmi = normalized_mutual_info_score(y_sample, cluster_labels)
sil = silhouette_score(X_umap, cluster_labels)

print(f"UMAP → KMeans (k={k})")
print(f"  Adjusted Rand Index : {ari:.4f}")
print(f"  Normalized MI         : {nmi:.4f}")
print(f"  Silhouette (UMAP)      : {sil:.4f}")

# --- Quick cross-tab to see dominant label per cluster ---
ct = pd.crosstab(cluster_labels, y_sample)
dominant = ct.idxmax(axis=1).rename("dominant_true_label")
sizes = ct.sum(axis=1).rename("cluster_size")
summary = pd.concat([dominant, sizes], axis=1).sort_index()
print("\nTop true label per cluster:\n", summary)

# --- Visual comparison plots ---
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plot by predicted clusters
axs[0].scatter(X_umap[:, 0], X_umap[:, 1], c=cluster_labels, s=6,
alpha=0.8)
axs[0].set_title("UMAP embedding colored by KMeans clusters")
axs[0].set_xlabel("UMAP-1"); axs[0].set_ylabel("UMAP-2")

# Plot by true labels
axs[1].scatter(X_umap[:, 0], X_umap[:, 1], c=y_sample, s=6, alpha=0.8)
axs[1].set_title("UMAP embedding colored by true labels")
axs[1].set_xlabel("UMAP-1"); axs[1].set_ylabel("UMAP-2")

plt.tight_layout()
plt.show()

```

```

UMAP → KMeans (k=10)
  Adjusted Rand Index : 0.4673
  Normalized MI       : 0.6080
  Silhouette (UMAP)   : 0.5165

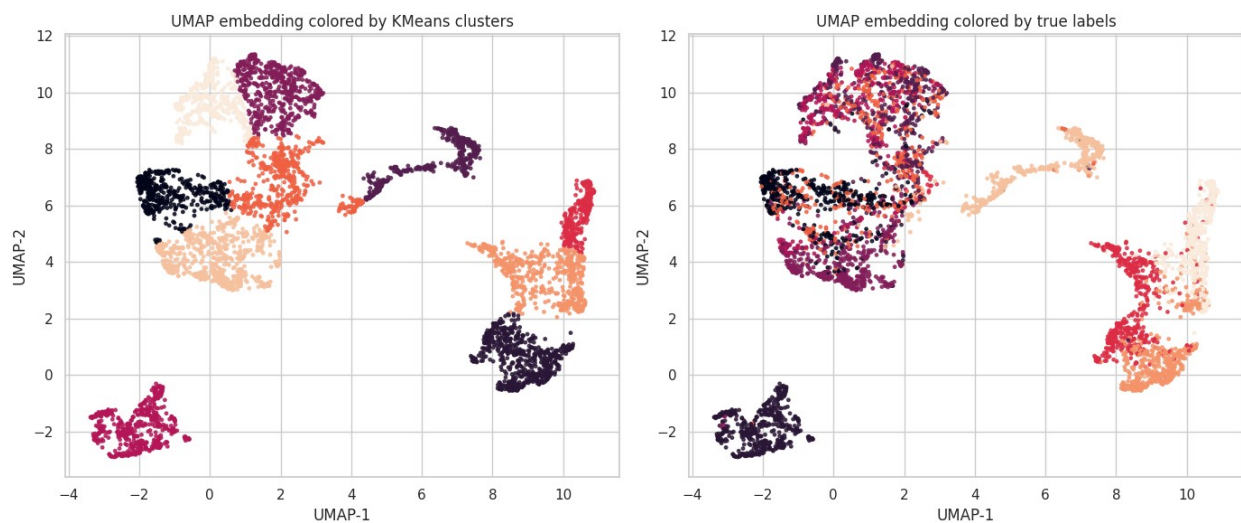
```

```

Top true label per cluster:
      dominant_true_label  cluster_size
row_0
0                        0           428
1                        7           637
2                        8           392
3                        2           595
4                        1           526
5                        9           286
6                        6           502
7                        5           573

```

8	3	644
9	4	417



► *Visual Comparison: UMAP vs PCA*

1) *PCA to 2D*

```
pca2 = PCA(n_components=2, random_state=42)
X_pca2_sample = pca2.fit_transform(X_sample)
```

```
plot_2D(X_pca2_sample, y_sample, "PCA (2D Projection)")
```

2) *Use existing UMAP embedding if available, else compute it*

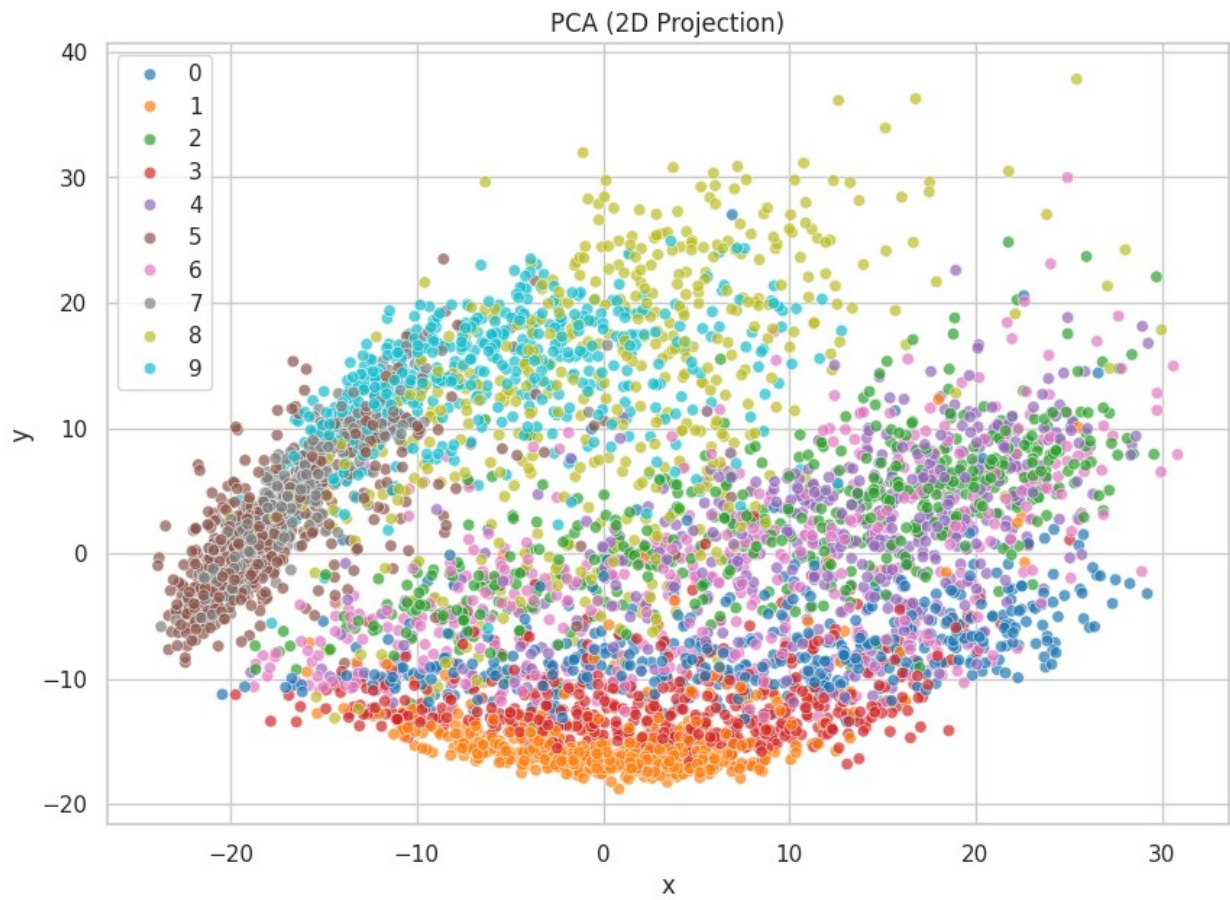
try:

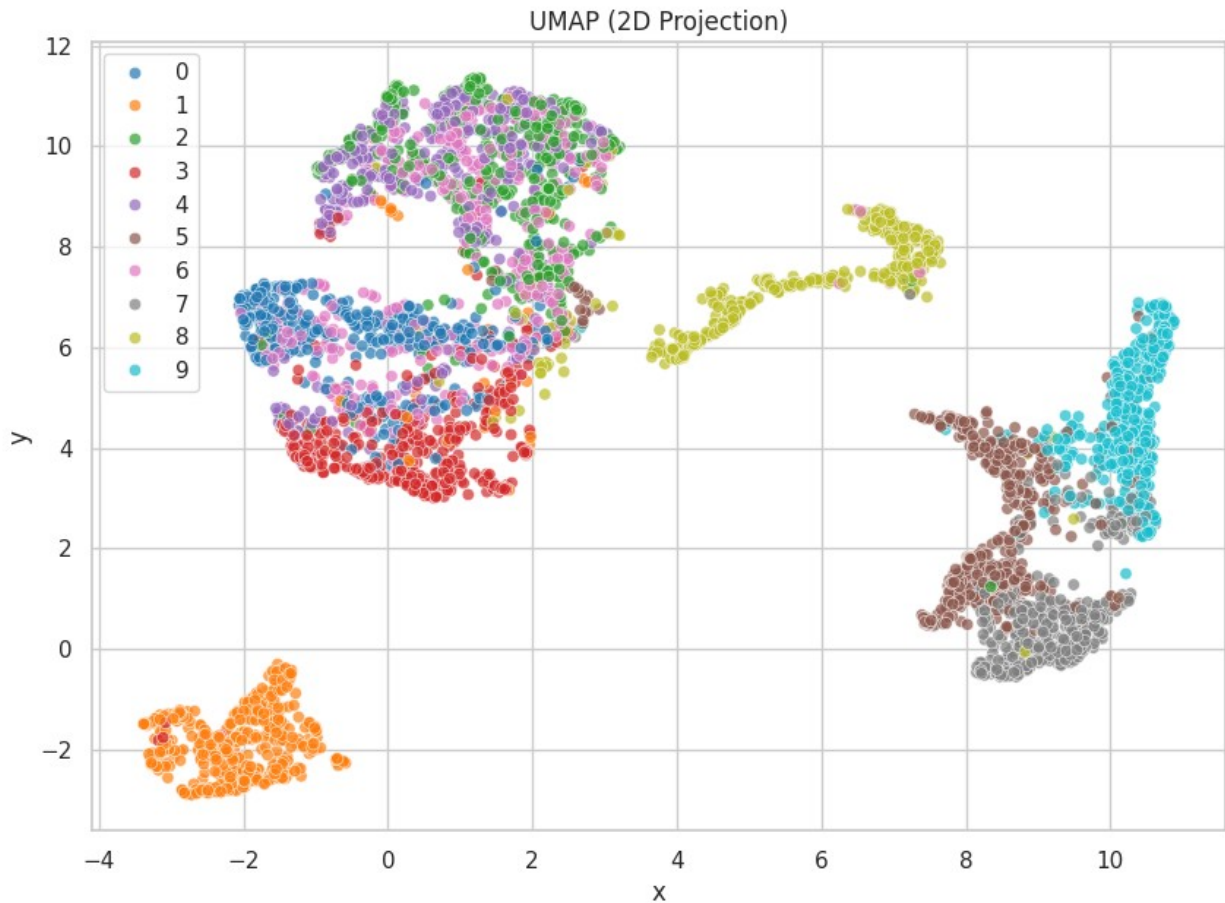
```
X_umap
```

except `NameError`:

```
    reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X_sample)
```

```
plot_2D(X_umap, y_sample, "UMAP (2D Projection)")
```





Code Explanation:

Experiments with UMAP first vary the `nneighbors` parameter to demonstrate the differences in research on how the size of the local neighborhood impacts the shape of the 2D embedding. It next feeds the features, which are UMAP-transformed, to KMeans clustering, assesses the quality of the clustering by ARI, NMI and silhouette score, and displays which of the true classes is the most common in each cluster to gain an understanding of how the clusters and labels are aligned. Visual comparisons between the UMAP embedding colored by predicted clusters and true labels are also made in the code to analyze the ability of UMAP to cluster the data well. Lastly, it directly compares UMAP and PCA through plotting both methods by embedding the resulting 2D space, showing that PCA only retains the structure of global variance, whereas UMAP can give more distinct and separated clusters of classes in the reduced dimension space.

Q. How UMAP Preserves Global Structure

UMAP maintains the global structure by simulating the dataset in the form of a weighted graph which reflects a local and extensive relationship between data points. It starts by creating a neighborhood graph with each node being linked to its closest neighbors with an edge weight being a measure of similarity. This creates a topological representation of the original high dimensional space that is fuzzy. Unlike the relationships among neighbors on a small scale, UMAP also takes into account additional relationships between the neighborhoods on a larger scale within the dataset. In the embedding phase, UMAP tries to preserve such a connectivity structure such that similar clusters in high-dimensional space are also close in the low

dimensional representation. This solution will enable UMAP to indicate not only close groupings but also approximate distances or relative positioning of various areas or clusters in the data. Consequently, UMAP is more likely to give embeddings in which the entire structure of clusters is better respected. This is unlike the approaches where the local form is so accentuated to the extent that the global form is distorted. UMAP uses local geometry, manifold approximation and global topological constraints to retain a more meaningful large scale sense of relationships between clusters, which is useful in interpretation and exploration.

Q.Why UMAP Is Faster Than t-SNE

UMAP is more efficient than t-SNE mainly due to the fact that it employs more effective mathematical models and optimization techniques. UMAP is based on approximate neighbor search algorithms to build a nearest-neighbor graph, which is much faster to compute than the greater computational distinctions of t-SNE that depends on pairwise comparisons at a distance. After making the graph, UMAP minimizes the low dimensional embedding with stochastic gradient descent that operates directly on the graph structure. It is also scalable to large datasets since the operations performed increase approximately in direct proportion to the size of the data sets. Conversely, a more computationally expensive objective is used by t-SNE which models pairwise probabilities between points and hence, its time complexity is quadratic or near quadratic. Even though t-SNE can be optimized using techniques like Barnes-Hut and FFT-based, they are still not as efficient as UMAP is at its core. Also, the formulation of UMAP enables convergence to occur more quickly on optimization, i.e. it finds useful embeddings in fewer steps. Such design options ensure that UMAP can be of significant use in large or high-dimensional data sets in which t-SNE can be slow or infeasible.

□ Augmentation

Coding:

Build a pipeline with 4 augmentations.

Apply augmentation only to class 9.

Show a grid of 10 random augmentations.

Theory:

What are the dangers of over-augmentation?

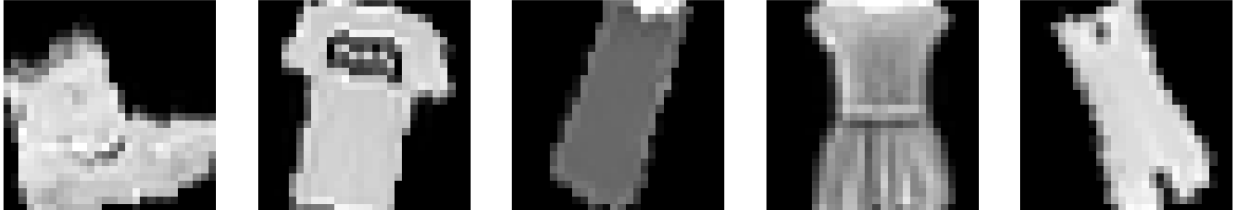
When to use GAN-based augmentation?

```
#Build a pipeline with 4 augmentations and preview results

# --- torchvision pipeline (exactly 4 augmentations) ---
aug_pipeline = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.7),
    transforms.RandomRotation(25),
    transforms.RandomResizedCrop(28, scale=(0.85, 1.0)),
    transforms.ColorJitter(brightness=0.35, contrast=0.35)
])
```

```
# Preview on the existing sample images (uses your show_augmented
helper)
show_augmented(pil_images, aug_pipeline , "TorchVision: 4-Aug
Pipeline")
```

TorchVision: 4-Aug Pipeline



```
# ▶ Apply augmentation only to class 9

# Class 9 = "Ankle boot"
target_class = 9

X_aug = []
y_aug = []

for img, label in zip(X_train, y_train):
    img_pil = Image.fromarray(img)

    if label == target_class:
        img_pil = aug_pipeline(img_pil)    # apply augmentation only to
class 9

        X_aug.append(np.array(img_pil))
        y_aug.append(label)

X_aug = np.array(X_aug)
y_aug = np.array(y_aug)

# Show a grid of 10 random augmentations
# Pick sample images only from class 9 (Ankle boot)
class9_images = X_train[y_train == 9]

# Select one random image to repeatedly augment
base_img =
Image.fromarray(class9_images[np.random.randint(len(class9_images))])

# Generate 10 augmented versions
augmented_samples = [aug_pipeline(base_img) for _ in range(10)]

# Display in a grid
plt.figure(figsize=(12, 3))
for i, img in enumerate(augmented_samples):
```

```
plt.subplot(1, 10, i+1)
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.suptitle("10 Random Augmentations (Class 9: Ankle Boot)")
plt.show()
```

10 Random Augmentations (Class 9: Ankle Boot)



Code Expalantion:

This code constructs a data augmentation pipeline consisting of four changes (horizontal flip, rotation, random resized crop as well as brightness/contrast adjustment) to add more visual diversity to the dataset. The pipeline is initially shown on sample images in order to demonstrate the effect of the augmentations on appearance. Thereafter, the augmentation is only done to class 9 (Ankle boot) in such a way that the class 9 is only expanded not to interfere with the other classes, this is handy when augmentation of classes is required to deal with the classes imbalance. The labels and augmented images are put in new arrays (Xaug and y_aug). Lastly, the code picks one random ankle-boot image and uses the augmentation pipeline perceiving ten visually different versions, presenting them in a grid to clearly show the ways in which the augmentation pipeline causes significant variations in the image, maintaining the identity of the original class.

Q.What are the dangers of over-augmentation?

The purpose of augmentation is to ensure that the models are more resilient by adding more variability to the training data but excessive augmentation can be counterproductive to learning. The effect of over-augmentation is that the critical aspects of the classes are misrepresented in the images and the model will have learned something that is not present in the real data. To illustrate, over- rotation, over-cropping or color adjustment can result in an image that is no longer similar to the original category and hence result in confusion during the training. In case the augmented data is not realistic, the model might not generalize well and instead, it will learn spurious or false representations. It may also result in over-augmentation that will waste time training in areas that it is not necessary to train. Also, when augmentation is administered unevenly across the classes, it can lead to a form of imbalance or bias in classification, which influences the accuracy of classification. Augmentation is aimed at bringing in natural and significant variation. Beyond the level of augmentation which is believable in that task or dataset, the overall effect is not a strengthening of performance, but a weakening. Thus, the strategies of augmentation should be carefully tuned, and the empirical validation of their effectiveness should be evaluated to ensure that a quality of learned representations is not damaged.

Q.When to use GAN-based augmentation?

GAN based augmentation can also be applied in the event that the dataset is small, skewed, or not diverse enough with regard to specific classes. The GANs are capable of producing new synthetic images, which are similar to actual ones, which assist in multiplying the number of under-represented categories without necessarily copying the already existing data. It has found application especially in medical imaging, remote sensing, rare object detection and application where new data with labels is either costly or inappropriate to gather. GANs are learners of the data distribution; thus, they can generate variations of the data distribution, which retain significant characteristics and generate plausible diversity. The GAN-based augmentation is necessary where other conventional methods of transformation such as flipping, rotation, or noise addition are not adequate to provide the difference required in the real world situation. Nevertheless, the application of GANs is to be used with caution since when GANs are not properly trained, they might generate low-quality or noisy images thus, deteriorating the performance of the models. Ideally, the samples generated by GAN have to be checked both visually and experimentally so that they will not corrupt the learning goal instead of supporting it.